

Name: Manahil Habib

Section: 5-B

Reg no: 2023-BSE-042

Lab Exam

Q1 – AWS IAM Setup Using AWS CLI and Console Verification (10 marks)

You are working as a junior cloud engineer. Your team needs an IAM setup for the **Software Engineering** group. Using the **AWS CLI** and then verifying in the **AWS Management Console**, complete the following.

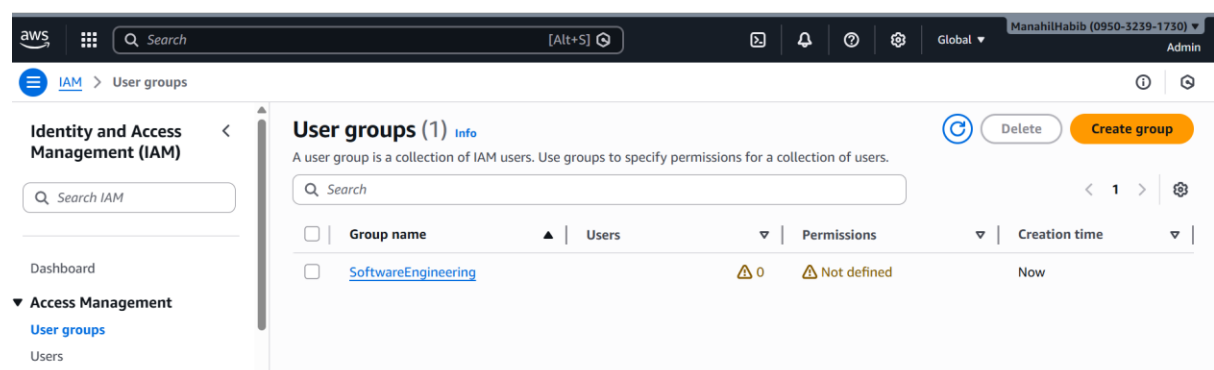
You must:

- Use **your own name** for the IAM user (for example: Ali, Sara, YOURNAME).
- Use **exactly** this group name: SoftwareEngineering.
- Take the specified CLI and console screenshots at each point.

1. Create IAM group SoftwareEngineering using AWS CLI

- Use the AWS CLI to **create** an IAM group named SoftwareEngineering.

```
@manahilhabib031 →/workspaces/labexam (main) $ aws iam create-group --group-name SoftwareEngineering
{
  "Group": {
    "Path": "/",
    "GroupName": "SoftwareEngineering",
    "GroupId": "AGPARMIC7HAZNZHFXYL4",
    "Arn": "arn:aws:iam::095032391730:group/SoftwareEngineering",
    "CreateDate": "2026-01-19T07:39:51+00:00"
  }
}
```



- Then use the AWS CLI to **view the group details**.

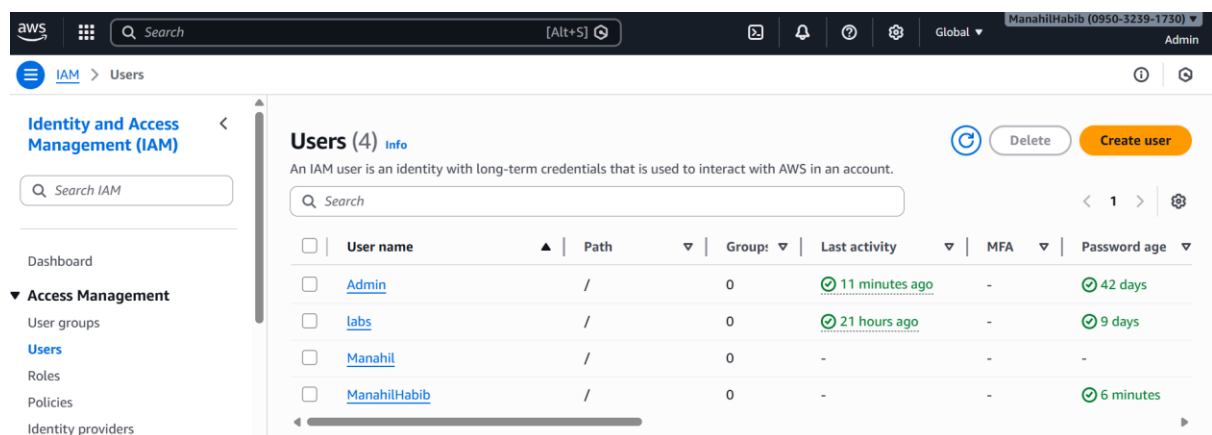
```
@manahilhabib031 →/workspaces/labexam (main) $ aws iam get-group --group-name SoftwareEngineering
{
  "Users": [],
  "Group": {
    "Path": "/",
    "GroupName": "SoftwareEngineering",
    "GroupId": "AGPARMIC7HAZNZHF5XYL4",
    "Arn": "arn:aws:iam::095032391730:group/SoftwareEngineering",
    "CreateDate": "2026-01-19T07:39:51+00:00"
  }
}
```

- The **group name** SoftwareEngineering, and
- The **group ARN**.

2. Create IAM user (your name) and view details

- Using the AWS CLI, **create an IAM user** whose username is based on your own name (for example, Ali, Sara, YOURNAME).

```
@manahilhabib031 →/workspaces/labexam (main) $ aws iam create-user --user-name Manahil
{
  "User": {
    "Path": "/",
    "UserName": "Manahil",
    "UserId": "AIDARMIC7HAZBQZTBDXD",
    "Arn": "arn:aws:iam::095032391730:user/Manahil",
    "CreateDate": "2026-01-19T07:41:53+00:00"
  }
}
```



The screenshot shows the AWS IAM console 'Users' page. The left sidebar contains navigation links for Identity and Access Management (IAM), Dashboard, Access Management, User groups, Users, Roles, Policies, and Identity providers. The main content area shows a list of users with columns for checkboxes, User name, Path, Group, Last activity, MFA, and Password age. The users listed are Admin, labs, Manahil, and ManahilHabib. The 'Manahil' user is selected, and its details are shown in the table.

	User name	Path	Group	Last activity	MFA	Password age
☐	Admin	/	0	✓ 11 minutes ago	-	✓ 42 days
☐	labs	/	0	✓ 21 hours ago	-	✓ 9 days
☐	Manahil	/	0	-	-	-
☐	ManahilHabib	/	0	-	-	✓ 6 minutes

- Use the AWS CLI to **view the user details**.
- The **username**, and

- The user ARN.

```
@manahilhabib031 →/workspaces/labexam (main) $ aws iam get-user --user-name Manahil
{
  "User": {
    "Path": "/",
    "UserName": "Manahil",
    "UserId": "AIDARMIC7HAZBQZTBTDXD",
    "Arn": "arn:aws:iam::095032391730:user/Manahil",
    "CreateDate": "2026-01-19T07:41:53+00:00"
  }
}
@manahilhabib031 →/workspaces/labexam (main) $
```

3. Add the IAM user to the SoftwareEngineering group

- Using the AWS CLI, add the IAM user (created above) to the SoftwareEngineering group.

```
@manahilhabib031 →/workspaces/labexam (main) $ aws iam add-user-to-group \
--user-name Manahil \
--group-name SoftwareEngineering
@manahilhabib031 →/workspaces/labexam (main) $
```

The screenshot shows the AWS IAM console interface. On the left is a navigation menu with 'Identity and Access Management (IAM)' selected. The main content area shows the 'SoftwareEngineering' group details. The 'Summary' tab is active, displaying the group name, creation time (January 19, 2026, 12:39 UTC+05:00), and ARN (arn:aws:iam::095032391730:group/SoftwareEngineering). Below this, the 'Users (1)' tab is selected, showing a table with one user, 'Manahil'. The table has columns for 'User name', 'Groups', 'Last activity', and 'Creation time'. The 'Manahil' user is listed with '1' group, 'None' last activity, and '2 minutes ago' creation time. There are 'Remove' and 'Add users' buttons at the top of the users list.

- Use the AWS CLI to **view the group membership** and confirm your user is a member.

```
@manahilhabib031 →/workspaces/labexam (main) $ aws iam get-group --group-name SoftwareEngineering
{
  "Users": [
    {
      "Path": "/",
      "UserName": "Manahil",
      "UserId": "AIDARMIC7HAZBQZTBDXD",
      "Arn": "arn:aws:iam::095032391730:user/Manahil",
      "CreateDate": "2026-01-19T07:41:53+00:00"
    }
  ],
  "Group": {
    "Path": "/",
    "GroupName": "SoftwareEngineering",
    "GroupId": "AGPARMIC7HAZNZHFSXYL4",
    ...skipping...
  }
}
```

```
@manahilhabib031 →/workspaces/labexam (main) $ aws iam get-group --group-name SoftwareEngineering
~
~
~
(END) ...skipping...
{
  "Users": [
    {
      "Path": "/",
      "UserName": "Manahil",
      "UserId": "AIDARMIC7HAZBQZTBDXD",
      "Arn": "arn:aws:iam::095032391730:user/Manahil",
      "CreateDate": "2026-01-19T07:41:53+00:00"
    }
  ],
  "Group": {
    "Path": "/",
    "GroupName": "SoftwareEngineering",
    "GroupId": "AGPARMIC7HAZNZHFSXYL4",
    "Arn": "arn:aws:iam::095032391730:group/SoftwareEngineering",
    "CreateDate": "2026-01-19T07:39:51+00:00"
  }
}
~
~
~
~
```

4. Attach AdministratorAccess managed policy to the SoftwareEngineering group

- Using the AWS CLI, **locate and identify** the AWS managed policy named AdministratorAccess, including its ARN (for example, by using `aws iam list-policies` or `aws iam get-policy` with the known ARN).

```
@manahilhabib031 →/workspaces/labexam (main) $ aws iam list-policies --scope AWS | grep AdministratorAccess
{"PolicyName": "AdministratorAccess",
 "Arn": "arn:aws:iam::aws:policy/AdministratorAccess",
 "PolicyName": "AdministratorAccess-Amplify",
 "Arn": "arn:aws:iam::aws:policy/AdministratorAccess-Amplify",
 "PolicyName": "AWSAuditManagerAdministratorAccess",
 "Arn": "arn:aws:iam::aws:policy/AWSAuditManagerAdministratorAccess",
 "PolicyName": "AdministratorAccess-AWSElasticBeanstalk",
 "Arn": "arn:aws:iam::aws:policy/AdministratorAccess-AWSElasticBeanstalk",
 "PolicyName": "AWSManagementConsoleAdministratorAccess",
 "Arn": "arn:aws:iam::aws:policy/job-function/AWSManagementConsoleAdministratorAccess",
@manahilhabib031 →/workspaces/labexam (main) $
```

```
@manahilhabib031 →/workspaces/labexam (main) $ aws iam get-policy \
--policy-arn arn:aws:iam::aws:policy/AdministratorAccess
{
  "Policy": {
    "PolicyName": "AdministratorAccess",
    "PolicyId": "ANPAIWMBCKSKIEE64ZLYK",
    "Arn": "arn:aws:iam::aws:policy/AdministratorAccess",
    "Path": "/",
    "DefaultVersionId": "v1",
    "AttachmentCount": 3,
    "PermissionsBoundaryUsageCount": 0,
    "IsAttachable": true,
    "Description": "Provides full access to AWS services and resources.",
    "CreateDate": "2015-02-06T18:39:46+00:00",
    "UpdateDate": "2015-02-06T18:39:46+00:00",
    "Tags": []
  }
}
@manahilhabib031 →/workspaces/labexam (main) $
```

- The **policy name** AdministratorAccess, and
- The **policy ARN**.
- Using the AWS CLI, **attach** the AdministratorAccess managed policy to the SoftwareEngineering group using its policy ARN.

```
@manahilhabib031 →/workspaces/labexam (main) $ aws iam attach-group-policy \
--group-name SoftwareEngineering \
--policy-arn arn:aws:iam::aws:policy/AdministratorAccess
```

5. List attached policies of the SoftwareEngineering group

- Using the AWS CLI, **list the policies** attached to the SoftwareEngineering group (for example, list-attached-group-policies).

```
@manahilhabib031 →/workspaces/labexam (main) $ aws iam list-attached-group-policies \
--group-name SoftwareEngineering
{
  "AttachedPolicies": [
    {
      "PolicyName": "AdministratorAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/AdministratorAccess"
    }
  ]
}
@manahilhabib031 →/workspaces/labexam (main) $
```

6. Verify IAM configuration in AWS Management Console

After completing all CLI steps above, log in to the **AWS Management Console** and verify that:

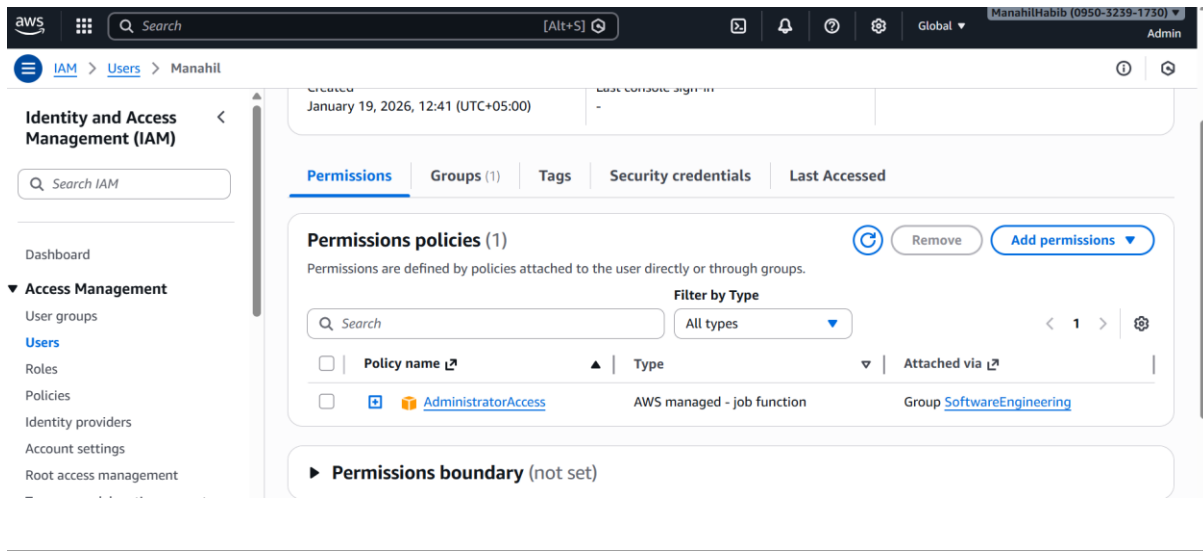
- The SoftwareEngineering group exists.
- Your IAM user exists and is part of that group.
- The group has the AdministratorAccess managed policy attached.

The first screenshot shows the 'User groups (1)' page in the AWS IAM console. It displays a table with one group, 'SoftwareEngineering', which has 1 user and the 'AdministratorAccess' policy attached. The group was created 13 minutes ago.

Group name	Users	Permissions	Creation time
SoftwareEngineering	1	Defined	13 minutes ago

The second screenshot shows the 'SoftwareEngineering' group details page. It includes a summary with the group name, creation time (January 19, 2026, 12:39 UTC+05:00), and ARN (arn:aws:iam::095032391730:group/SoftwareEngineering). Below the summary, the 'Users (1)' tab is active, showing a table with one user, 'Manahil', who has no groups and was created 9 minutes ago.

User name	Groups	Last activity	Creation time
Manahil	1	None	9 minutes ago



Q2 – Terraform Lab: Simple AWS Environment with Nginx over HTTPS (30 marks)

You are working as an infrastructure engineer and must provision a simple application environment in AWS using **Terraform only**. No resources may be created manually in the AWS console.

You will create Terraform configuration and shell script files to deploy the required infrastructure.

You must:

- Write all Terraform files and scripts yourself.
- Use a data "http" source and a locals block for your /32 IP.

1. Configure the AWS provider

Create the provider configuration so that the AWS provider reads from:

- ~/.aws/config
- ~/.aws/credentials

(You may use the default profile or a named profile.)

- **Save screenshot as:** q2_provider.png — editor or terminal view clearly showing the **provider** block in your .tf file (e.g., main.tf).

```
GNU nano 7.2 main.tf
provider "aws" {
  region = "me-central-1"
}
```

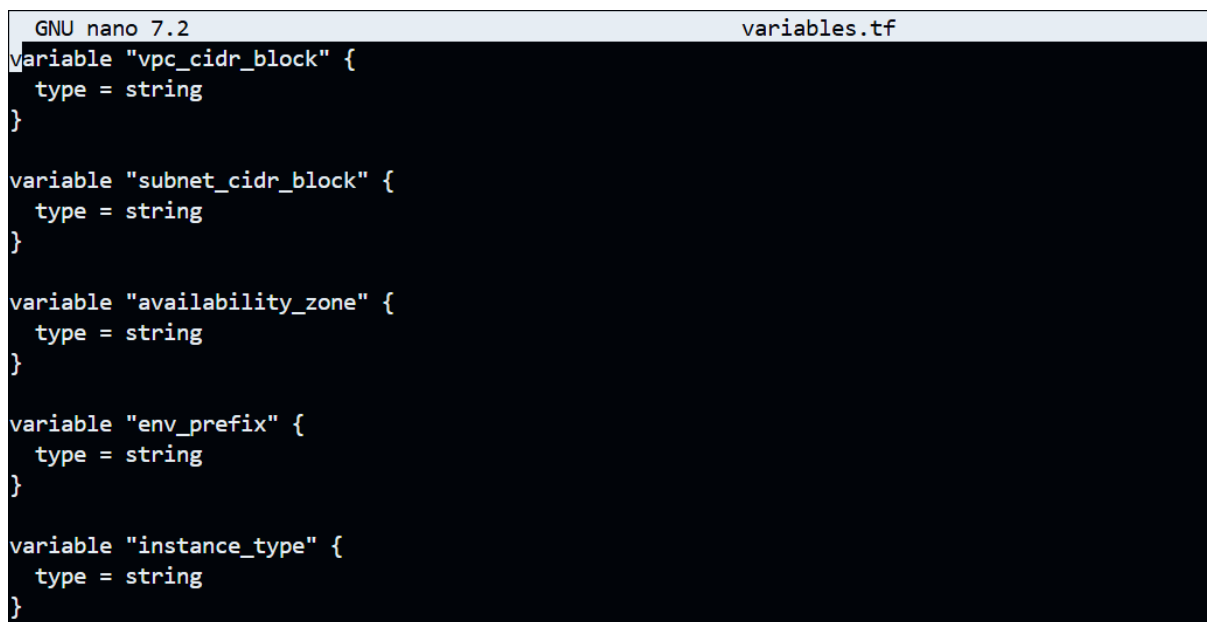
2. Define input variables

Define the following input variables:

- vpc_cidr_block
- subnet_cidr_block
- availability_zone
- env_prefix
- instance_type

Place them in variables.tf or any .tf file.

- **Save screenshot as:** q2_variables.png — editor view showing all five variable definitions.



```
GNU nano 7.2 variables.tf
variable "vpc_cidr_block" {
  type = string
}

variable "subnet_cidr_block" {
  type = string
}

variable "availability_zone" {
  type = string
}

variable "env_prefix" {
  type = string
}

variable "instance_type" {
  type = string
}
```

3. Create VPC and subnet

Using Terraform:

- Create a **VPC** (e.g., aws_vpc.myapp_vpc) using var.vpc_cidr_block as CIDR.
Tag:

Name = "<env_prefix>-vpc"

- Create a **subnet** attached to this VPC with:
 - var.subnet_cidr_block as CIDR,
 - var.availability_zone as AZ,
 - Tag:

Name = "<env_prefix>-subnet-1"

- **Save screenshot as:** q2_vpc_subnet.png — editor view showing VPC and subnet resources with tags.

A screenshot of a terminal window with a dark background. The title bar at the top shows 'GNU nano 7.2' on the left and 'main.tf *' on the right. The terminal displays Terraform configuration code for an AWS VPC and Subnet. The code defines two resources: 'aws_vpc' named 'myapp_vpc' and 'aws_subnet' named 'myapp_subnet'. Both resources include tags with names derived from an environment variable 'env_prefix'. The VPC tag name is '\${var.env_prefix}-vpc' and the Subnet tag name is '\${var.env_prefix}-subnet-1'.

```
GNU nano 7.2                               main.tf *
provider "aws" {
  region = "me-central-1"
}

resource "aws_vpc" "myapp_vpc" {
  cidr_block = var.vpc_cidr_block

  tags = {
    Name = "${var.env_prefix}-vpc"
  }
}

resource "aws_subnet" "myapp_subnet" {
  vpc_id            = aws_vpc.myapp_vpc.id
  cidr_block        = var.subnet_cidr_block
  availability_zone = var.availability_zone

  tags = {
    Name = "${var.env_prefix}-subnet-1"
  }
}
```

4. Create Internet Gateway and configure default route table

- Create an **Internet Gateway** attached to the VPC and tag it:

Name = "<env_prefix>-igw"

- Use the VPC's **default route table** to add a route 0.0.0.0/0 pointing to this IGW, and tag:

Name = "<env_prefix>-rt"

- **Save screenshot as:** q2_igw_route_table.png — editor view showing the IGW and default route table configuration, including the 0.0.0.0/0 route and tags.

```
GNU nano 7.2                                main.tf *
    Name = "${var.env_prefix}-subnet-1"
  }
}
resource "aws_internet_gateway" "myapp_igw" {
  vpc_id = aws_vpc.myapp_vpc.id

  tags = {
    Name = "${var.env_prefix}-igw"
  }
}

resource "aws_default_route_table" "default_rt" {
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.myapp_igw.id
  }

  tags = {
    Name = "${var.env_prefix}-rt"
  }
}
```

5. Discover public IP and compute /32 CIDR using data + locals

- Use a Terraform data "http" block to query your **current public IP** (e.g., <https://icanhazip.com>).
- Use a locals block to:
 - Take the raw HTTP response body,
 - Strip trailing newline(s),
 - Append /32 to create a single-host CIDR locals.my_ip.
- **Save screenshot as:** q2_http_and_locals.png — editor view showing the data "http" block and locals block with locals.my_ip.

```
GNU nano 7.2                                main.tf *
    Name = "${var.env_prefix}-igw"
  }
}

resource "aws_default_route_table" "default_rt" {
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.myapp_igw.id
  }

  tags = {
    Name = "${var.env_prefix}-rt"
  }
}

data "http" "myip" {
  url = "https://icanhazip.com"
}

locals {
  my_ip = "${chomp(data.http.myip.response_body)}/32"
}
```

6. Configure the default security group in the VPC

Manage the **default security group** of your VPC with:

- Ingress:
 - SSH (TCP 22) from locals.my_ip.
 - HTTP (TCP 80) from 0.0.0.0/0.
 - HTTPS (TCP 443) from 0.0.0.0/0.
- Egress:
 - All outbound traffic to 0.0.0.0/0.

Tag:

Name = "<env_prefix>-default-sg"

- **Save screenshot as:** q2_default_sg.png — editor view showing the default SG Terraform resource with all rules and tag.

GNU nano 7.2

main.tf *

```
my_ip = "${chomp(data.http.myip.response_body)}/32"
}
resource "aws_default_security_group" "default_sg" {
  vpc_id = aws_vpc.myapp_vpc.id

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = [local.my_ip]
  }

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port = 443
    to_port   = 443
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

GNU nano 7.2

main.tf *

```
protocol = "tcp"
cidr_blocks = [local.my_ip]
}

ingress {
  from_port = 80
  to_port   = 80
  protocol  = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

ingress {
  from_port = 443
  to_port   = 443
  protocol  = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

egress {
  from_port = 0
  to_port   = 0
  protocol  = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}
```

```
egress {
  from_port    = 0
  to_port      = 0
  protocol     = "-1"
  cidr_blocks  = ["0.0.0.0/0"]
}

tags = {
  Name = "${var.env_prefix}-default-sg"
}
}
```

Help Write Out Where Is Cut Execu
Exit Read File Replace Paste Justi

7. Create an AWS key pair for SSH

Create an `aws_key_pair` resource:

- Key name: `serverkey`
- Public key: `~/.ssh/id_ed25519.pub`
- **Save screenshot as:** `q2_keypair.png` — editor view showing the key pair resource referencing `~/.ssh/id_ed25519.pub`.

```
GNU nano 7.2 main.tf *

ingress {
  from_port    = 443
  to_port      = 443
  protocol     = "tcp"
  cidr_blocks  = ["0.0.0.0/0"]
}

egress {
  from_port    = 0
  to_port      = 0
  protocol     = "-1"
  cidr_blocks  = ["0.0.0.0/0"]
}

tags = {
  Name = "${var.env_prefix}-default-sg"
}

resource "aws_key_pair" "serverkey" {
  key_name     = "serverkey"
  public_key   = file("~/ssh/id_ed25519.pub")
}
```

8. Create the EC2 instance resource

Launch an EC2 instance that:

- Uses an Amazon Linux 2023 AMI (hard-coded ID).
- Uses `var.instance_type`.
- Is in the subnet you created.
- Attaches the **default security group** from step 6.
- Uses `var.availability_zone`.
- Has a public IP address.
- Uses key pair `serverkey`.
- Uses `user_data` from a local file `entry-script.sh`.
- Is tagged:

Name = "<env_prefix>-ec2-instance"

- **Save screenshot as:** `q2_ec2_resource.png` — editor view showing the EC2 instance resource with subnet, SG, key pair, `user_data`, and tag.

```
resource "aws_instance" "myapp_ec2" {
  ami                = "ami-0a0c8eebcdd6dcdbd0"
  instance_type      = var.instance_type
  subnet_id          = aws_subnet.myapp_subnet.id
  vpc_security_group_ids = [aws_default_security_group.default_sg.id]
  availability_zone   = var.availability_zone
  associate_public_ip_address = true
  key_name            = aws_key_pair.serverkey.key_name
  user_data           = file("entry-script.sh")

  tags = {
    Name = "${var.env_prefix}-ec2-instance"
  }
}
```

9. Create `entry-script.sh` to configure Nginx + HTTPS

In the same directory as your `.tf` files, create `entry-script.sh` that:

- Installs **Nginx**.
- Generates/configures a **self-signed TLS certificate**.
- Configures Nginx to:
 - Listen on **HTTPS (443)** using that certificate,

- Serve **HTTP (80)** (redirect to HTTPS or serve a page).
- Enables Nginx on boot and ensures it is running after the script.
- Serves a page that includes:
 - Your **name**, and
 - The word **“Terraform”** (e.g. This is YOURNAME's Terraform environment.).
- **Save screenshot as:** q2_entry_script.png — editor or terminal view showing the full content of entry-script.sh.

```
GNU nano 7.2 entry-script.sh *
#!/bin/bash

dnf update -y
dnf install nginx openssl -y

mkdir -p /etc/nginx/ssl

openssl req -x509 -nodes -days 365 \
-newkey rsa:2048 \
-keyout /etc/nginx/ssl/nginx.key \
-out /etc/nginx/ssl/nginx.crt \
-subj "/CN=localhost"

cat <<EOF > /usr/share/nginx/html/index.html
<h1>This is Manahil's Terraform environment</h1>
EOF

systemctl enable nginx
systemctl start nginx
```

10. Add Terraform output for public IP

Add an output (e.g. ec2_public_ip) to print the instance’s public IP after terraform apply.

- **Save screenshot as:** q2_output_block.png — editor view showing the Terraform output exposing the EC2 public IP.

```
GNU nano 7.2 outputs.tf *
output "ec2_public_ip" {
  value = aws_instance.myapp_ec2.public_ip
}
```

11. Set variable values for apply time

When running terraform apply, use:

vpc_cidr_block = "10.0.0.0/16"

subnet_cidr_block = "10.0.10.0/24"

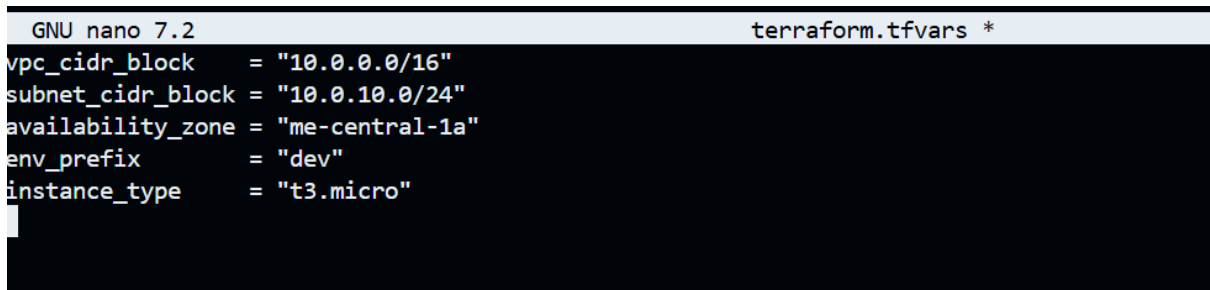
availability_zone = "me-central-1a"

env_prefix = "dev"

instance_type = "t3.micro"

(via terraform.tfvars, -var flags, or defaults)

- **Save screenshot as:** q2_tfvars_or_vars.png — editor view of terraform.tfvars or CLI evidence that these values are used.



```
GNU nano 7.2 terraform.tfvars *
vpc_cidr_block = "10.0.0.0/16"
subnet_cidr_block = "10.0.10.0/24"
availability_zone = "me-central-1a"
env_prefix = "dev"
instance_type = "t3.micro"
```

12. Run Terraform commands and capture outputs

From the Terraform project directory:

terraform init

- **Save screenshot as:** q2_terraform_init.png — terminal showing terraform init and successful initialization.

```
@manahilhabib031 →/workspaces/labexam/Lab_exam (main) $ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Finding latest version of hashicorp/http...
- Installing hashicorp/aws v6.28.0...
- Installed hashicorp/aws v6.28.0 (signed by HashiCorp)
- Installing hashicorp/http v3.5.0...
- Installed hashicorp/http v3.5.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@manahilhabib031 →/workspaces/labexam/Lab_exam (main) $
```

terraform plan

- **Save screenshot as:** q2_terraform_plan.png — terminal showing terraform plan and part of the plan.

```
@manahilhabib031 →/workspaces/labexam/Lab_exam (main) $ terraform plan
+ enable_network_address_usage_metrics = (known after apply)
+ id                                   = (known after apply)
+ instance_tenancy                     = "default"
+ ipv6_association_id                  = (known after apply)
+ ipv6_cidr_block                       = (known after apply)
+ ipv6_cidr_block_network_border_group = (known after apply)
+ main_route_table_id                  = (known after apply)
+ owner_id                             = (known after apply)
+ region                               = "me-central-1"
+ tags                                 = {
+   "Name" = "dev-vpc"
+ }
+ tags_all                             = {
+   "Name" = "dev-vpc"
+ }
}

Plan: 7 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ ec2_public_ip = (known after apply)

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions
run "terraform apply" now.
@manahilhabib031 →/workspaces/labexam/Lab_exam (main) $
```

terraform apply

(You may use -auto-approve.)

- **Save screenshot as:** q2_terraform_apply.png — terminal showing terraform apply and successful resource creation.

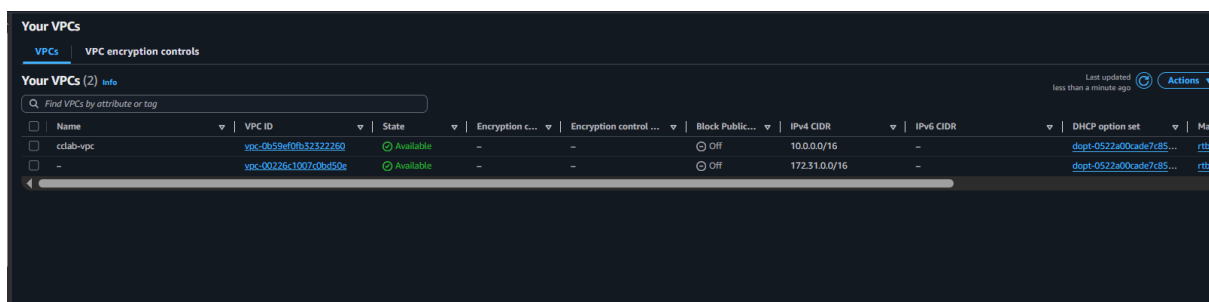
terraform output

- **Save screenshot as:** q2_terraform_output.png — terminal showing outputs including the EC2 public IP.

13. Verify Terraform resources in AWS console

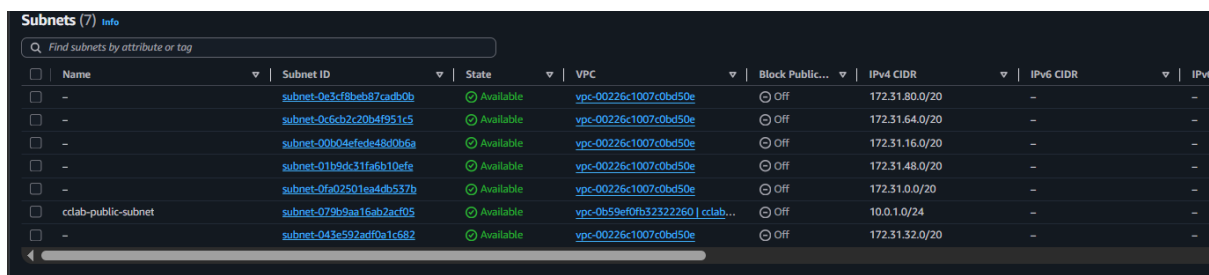
Use the AWS console to confirm:

- **VPC and Subnet**
 - **Save screenshot as:** q2_console_vpc.png — VPC with CIDR 10.0.0.0/16 and tag Name = dev-vpc.



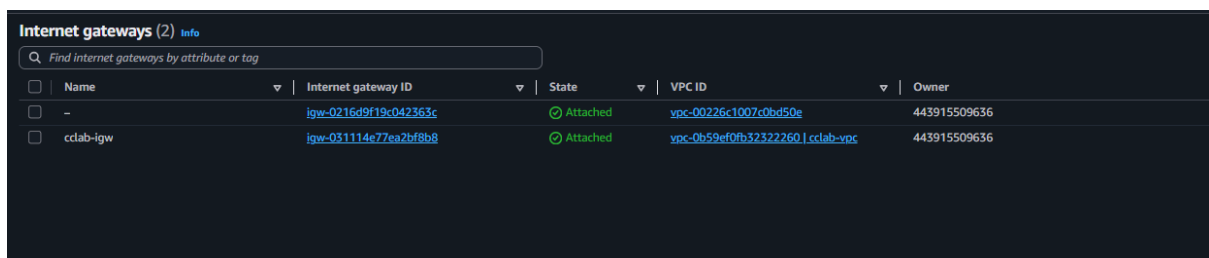
Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv4 CIDR	IPv6 CIDR	DHCP option set	Ma
cclab-vpc	vpc-0b59ef0fb32322260	Available	-	-	Off	10.0.0.0/16	-	dopt-0522a00cade7c85...	rtb...
-	vpc-00226c1007c0bd50e	Available	-	-	Off	172.31.0.0/16	-	dopt-0522a00cade7c85...	rtb...

- **Save screenshot as:** q2_console_subnet.png — subnet with CIDR 10.0.10.0/24, AZ me-central-1a, tag Name = dev-subnet-1.



Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 CIDR	IPv6
-	subnet-0e3cf8beb87cadb0b	Available	vpc-00226c1007c0bd50e	Off	172.31.80.0/20	-	-
-	subnet-0c6cb2c20b4f951c5	Available	vpc-00226c1007c0bd50e	Off	172.31.64.0/20	-	-
-	subnet-00b04efede48d0b6a	Available	vpc-00226c1007c0bd50e	Off	172.31.16.0/20	-	-
-	subnet-01b9dc31fa6b10efe	Available	vpc-00226c1007c0bd50e	Off	172.31.48.0/20	-	-
-	subnet-0fa02501ea4db537b	Available	vpc-00226c1007c0bd50e	Off	172.31.0.0/20	-	-
cclab-public-subnet	subnet-079b9aa16ab2acf05	Available	vpc-0b59ef0fb32322260 cclab...	Off	10.0.1.0/24	-	-
-	subnet-043e592adff0a1c682	Available	vpc-00226c1007c0bd50e	Off	172.31.32.0/20	-	-

- **Internet Gateway and Route Table**
 - **Save screenshot as:** q2_console_igw.png — IGW attached to the VPC with tag Name = dev-igw.



Name	Internet gateway ID	State	VPC ID	Owner
-	igw-0216cd9f19c042363c	Attached	vpc-00226c1007c0bd50e	443915509636
cclab-igw	igw-031114e77ea2bf8b8	Attached	vpc-0b59ef0fb32322260 cclab-vpc	443915509636

- **Save screenshot as:** q2_console_route_table.png — route table with tag Name = dev-rt and route 0.0.0.0/0 to the IGW.

Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC	Owner ID
-	rtb-01edc34018439038d	-	-	Yes	vpc-00226c1007c0bd50e	443915509636
-	rtb-008ef95a46cc62b5d	-	-	Yes	vpc-0b59ef0b32322260 cclab...	443915509636
cclab-public-rt	rtb-06801f3e712df5c7b	subnet-079b9aa16ab2acf...	-	No	vpc-0b59ef0b32322260 cclab...	443915509636

- **Security Group**
 - **Save screenshot as:** q2_console_sg.png — default SG showing:
 - SSH (22) from your /32 host,
 - HTTP (80) from 0.0.0.0/0,
 - HTTPS (443) from 0.0.0.0/0,
 - Egress all to 0.0.0.0/0,
 - Tag Name = dev-default-sg.
- **EC2 instance**
 - **Save screenshot as:** q2_console_ec2.png — instance:
 - In subnet 10.0.10.0/24, AZ me-central-1a,
 - With a public IP,
 - Using key pair serverkey,
 - Tagged Name = dev-ec2-instance.

14. Verify HTTPS access from browser

From your local machine:

https://<public-ip-of-instance>

Accept any self-signed cert warning and verify the page content.

- **Save screenshot as:** q2_https_browser.png — browser screenshot showing:
 - Address bar https://<public-ip-of-instance>,

Not secure <https://3.28.207.199>



Your connection is not private

Attackers might be trying to steal your information from **3.28.207.199** (for example, passwords, messages or credit cards). [Learn more about this warning](#)

NET:ERR_CERT_AUTHORITY_INVALID

Turn on enhanced protection to get Chrome's highest level of security

Advanced

Back to safety

- Nginx page with your **name** and **“Terraform”**.

Not secure <https://3.28.207.199>

Terraform .

Q3 – Ansible Playbook for EC2 Web Server Using Q2 Instance (10 marks)

You are working as a DevOps engineer. A previous solution used a shell script on an Amazon Linux EC2 instance to:

- Update the system,
- Install and start Apache HTTPD,
- Fetch instance metadata (public IP and public hostname) using **IMDSv2**,
- Print the public IP,
- Restart the HTTPD service.

Your task is to convert this logic into an **Ansible-based** solution targeting the EC2 instance created in **Step 2 (Q2)**.

If Nginx is running from Q2's user data, you must **stop/disable or uninstall nginx** so that httpd can bind to port 80.

1. Create Ansible inventory file hosts

In a directory such as Lab_exam/ansible/, create hosts that:

- Defines group ec2 with your Q2 instance's **public IP**.
- Sets group vars:
 - `ansible_user = ec2-user`
 - `ansible_ssh_private_key_file = ~/.ssh/id_ed25519`
 - `ansible_ssh_common_args` to disable strict host key checking (`-o StrictHostKeyChecking=no`).
- **Save screenshot as:** q3_hosts.png — editor or terminal view showing the complete hosts file (group, IP, vars).

```
GNU nano 7.2                                hosts *
[ec2]
40.172.151.106
[ec2:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/id_ed25519
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

2. Create project-level ansible.cfg

In the same directory, create ansible.cfg that:

- Disables host key checking.
- Sets remote Python interpreter to `/usr/bin/python3`.
- (Optional) Sets inventory = `./hosts`.
- **Save screenshot as:** q3_ansible_cfg.png — editor view showing ansible.cfg with required settings.

```
GNU nano 7.2                                ansible.cfg *
[defaults]
host_key_checking = False
inventory = ./hosts
remote_python_interpreter = /usr/bin/python3
```

3. Create Ansible playbook (e.g. my-playbook.yml)

Create my-playbook.yml that:

- Targets group ec2.
- Uses become: true.
- Performs actions equivalent to the shell script:
 - Update system packages.
 - Stop/disable or uninstall nginx if present (to free port 80).
 - Install httpd.
 - Start and enable httpd service.
 - Get IMDSv2 token using uri:
 - URL: `http://169.254.169.254/latest/api/token`
 - Method: PUT
 - Header: `X-aws-ec2-metadata-token-ttl-seconds: 21600`
 - Use token to fetch:
 - `http://169.254.169.254/latest/meta-data/public-ipv4`
 - `http://169.254.169.254/latest/meta-data/public-hostname`
 - Print **public IP** with debug.
 - Restart httpd.
- **Save screenshot as:** q3_playbook.png — editor view showing the full contents of my-playbook.yml.
-
-

```

@manahilhabib031 →/workspaces/labexam/Lab_exam/ansible (main) $ cat my-playbook.yml
method: PUT
headers:
  X-aws-ec2-metadata-token-ttl-seconds: 21600
return_content: yes
register: imds_token

- name: Get public IPv4 using IMDSv2
  uri:
    url: http://169.254.169.254/latest/meta-data/public-ipv4
    method: GET
    headers:
      X-aws-ec2-metadata-token: "{{ imds_token.content }}"
    return_content: yes
  register: public_ip

- name: Get public hostname using IMDSv2
  uri:
    url: http://169.254.169.254/latest/meta-data/public-hostname
    method: GET
    headers:
      X-aws-ec2-metadata-token: "{{ imds_token.content }}"
    return_content: yes
  register: public_hostname

- name: Print public IP
  debug:
    msg: "EC2 Public IP: {{ public_ip.content }}"

- name: Restart httpd
  systemd:
    name: httpd
    state: restarted

```

4. Run the Ansible playbook

Execute:

ansible-playbook -i hosts my-playbook.yml

(or without -i if your ansible.cfg sets the inventory.)

- **Save screenshot as:** q3_play_run.png — terminal showing:
 - The ansible-playbook command,

```

@manahilhabib031 →/workspaces/labexam/Lab_exam/ansible (main) $ ansible-playbook my-playbook.yml
[WARNING]: Ansible is being run in a world writable directory (/workspaces/labexam/Lab_exam/ansible), ignoring it as an ansible.cfg source. For more information see https://docs.ansible.com/ansible/devel/reference_appendices/config.html#cfg-in-world-writable-dir
[WARNING]: No inventory was parsed, only implicit localhost is available
[WARNING]: provided hosts list is empty, only localhost is available. Note that the implicit localhost does not match 'all'
[WARNING]: Could not match supplied host pattern, ignoring: ec2

PLAY [Configure EC2 web server] *****
skipping: no hosts matched

PLAY RECAP *****

@manahilhabib031 →/workspaces/labexam/Lab_exam/ansible (main) $

```

- All tasks (update, nginx handling, httpd, IMDSv2, debug of public IP),
- Successful completion.

5. Verify HTTP access

From your local machine:

`http://<public-ip-ec2>`

Confirm Apache (httpd) is responding.

- **Save screenshot as (optional):** q3_http_browser.png — browser or curl output showing HTTP access to `http://<public-ip-ec2>`.



Apache Web Server via Ansible

Public IP: 3.28.207.199

Files Required (Q3) in Lab_exam repo, e.g. Lab_exam/ansible/:

- hosts
- ansible.cfg
- my-playbook.yml

Screenshots Required:

- q3_hosts.png
 - q3_ansible_cfg.png
 - q3_playbook.png
 - q3_play_run.png
 - q3_http_browser.png (recommended)
-

Cleanup (ungraded)

After collecting all evidence:

1. From your Terraform project directory (Q2):

`terraform destroy -auto-approve`

- **Save screenshot as (recommended):** cleanup_terraform_destroy.png — terminal showing destroy output.
2. In AWS console, verify that no lab-related EC2 instances remain running.
 - **Save screenshot as (recommended):** cleanup_ec2_console.png — EC2 console showing no remaining lab instances.

(If your instructor gives different cleanup instructions, follow those.)